0.6.0 of the Solidity language. You specify the version number with the `pragma` directive. After the `pragma` directive, the `contract` keyword and the name of the contract (in this case, Counter) begin the definition of the contract code.

> **NOTE** At the time of completion of this book, the latest version of Solidity was 6.0. This version is used for examples. You'll have to make any minor changes that may be required for later versions when you read this book.

Next, you define the data components of the smart contract. The data types in Solidity are similar to those in any high-level language. In this case, the `uint` (unsigned integer) data type is used to define the identifier that stores the value of the counter. The `uint` type in the blockchain realm differs significantly from the integer data in mainstream computing: it's a 256-bit value as opposed to 64-bit in a general-purpose language. `uint`, `int`, `int256`, and `uint256` are aliases of one another.

Listing 2.1 Solidity code for Counter smart contract (Counter.sol)

```
pragma solidity ^0.6.0;
// imagine a big integer counter that the whole world could share
contract Counter {
    uint value;                                          ❶
    function initialize (uint x) public {               ❷
        value = x;

    }

    function get() view public returns (uint) {         ❷
        return value;
    }

    function increment (uint n) public {                ❷
        value = value + n;
```